Probabilistic Graphical Models

Causal Bayesian Networks
Guidelines and Rubrics
for Presentation and Report

Semester 1, 2026–2027

Instructor: Nguyen Chi Thien, Ph.D.

Outline

1. Synthetic benchmark for checking do-operator implementation
2. UCI application with B0, H1 and H2
3. Controlled comparisons and A1–A3 ablations
4. Presentation and report rubrics (10 points each)
5. Detailed implementation and submission guide (slides 38–62)

Synthetic benchmark: tutoring and exam success

Question: How would exam success change if all students received tutoring?
T: tutoring participation (0 = no, 1 = yes).
Y: exam success (0 = fail, 1 = pass).
Z: prior preparation (0 = low, 1 = high), measured before tutoring.
Use this synthetic binary model only to verify implementation against a known causal effect.
The main application must use a UCI tabular dataset.

Causal graph and assumptions

Graph: $Z \to T, \quad Z \to Y, \quad T \to Y$. Z is a common cause of T and Y.
Factorization: $P(Z, T, Y) = P(Z)\, P(T \mid Z)\, P(Y \mid T, Z)$.
Causal interpretation requires defensible directions and stable mechanisms under intervention.
Assume no unmeasured confounding, consistency and positivity: $0 < P(T = t \mid Z = z) < 1$.
A structure learned from observational data alone does not establish causal directions.

Sample conditional probability tables

Quantity	Condition	Probability
$P(Z = 1)$	Prior preparation	0.50
$P(T = 1 \mid Z)$	$Z = 0$ / $Z = 1$	0.20 / 0.80
$P(Y = 1 \mid T, Z)$	$T = 0,\ Z = 0$	0.10
$P(Y = 1 \mid T, Z)$	$T = 0,\ Z = 1$	0.50
$P(Y = 1 \mid T, Z)$	$T = 1,\ Z = 0$	0.40
$P(Y = 1 \mid T, Z)$	$T = 1,\ Z = 1$	0.80

For every binary variable, $P(\text{value} = 0 \mid \text{parents}) = 1 - P(\text{value} = 1 \mid \text{parents})$.

Observation and intervention

Observation: $P(Y = 1 \mid T = t) = \sum_z P\,(Y = 1 \mid t, z)P(z \mid t)$.
Intervention $\mathrm{do}(T = t)$ fixes T and removes the edge $Z \to T$.
Truncated factorization: $P\big(z, y \mid \mathrm{do}(T = t)\big) = P(z)P(y \mid t, z)$.
Therefore $P\big(Y = 1 \mid \mathrm{do}(T = t)\big) = \sum_z P\,(Y = 1 \mid t, z)P(z)$.
Risk difference (ATE for binary Y): $P\big(Y = 1 \mid \mathrm{do}(T = 1)\big) - P\big(Y = 1 \mid \mathrm{do}(T = 0)\big)$.

Exact reference values

Quantity	$T = 0$	$T = 1$	Difference $(1 - 0)$
$P(Z = 1 \mid T = t)$	0.20	0.80	0.60
$P(Y = 1 \mid T = t)$	0.18	0.72	0.54
$P\big(Y = 1 \mid \mathrm{do}(T = t)\big)$	0.30	0.60	0.30

Example: $P\big(Y = 1 \mid \mathrm{do}(T = 1)\big) = 0.40{\times}0.50 + 0.80{\times}0.50 = 0.60$. These are population values, not measured software outputs.

Synthetic benchmark: notebook workflow

1. Generate data, record the seed and check counts in all (Z, T) groups.
2. Specify the causal DAG, explain assumptions and fit BN parameters.
3. Query observational probabilities and compute the adjusted do probabilities.
4. Use DoWhy to identify the effect and estimate the same ATE.
5. Compare against the known population values and run refutation tests.
6. Repeat across seeds and sample sizes. Explain estimation uncertainty.
7. Simulate atomic interventions with DoWhy GCM and compare results.

Sample code: data generation

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(42)
n = 20000
z = rng.binomial(1, 0.5, n)
t = rng.binomial(1, 0.2 + 0.6*z)
y = rng.binomial(1, 0.1 + 0.3*t + 0.4*z)
df = pd.DataFrame({"Z": z, "T": t, "Y": y})
edges = [("Z", "T"), ("Z", "Y"), ("T", "Y")]
print(df.groupby(["Z", "T"]).size())
```

Each draw uses fresh random noise. The generating probabilities match the reference CPDs.

Sample code: BN fitting and do query

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import BayesianEstimator
from pgmpy.inference import VariableElimination
bn = DiscreteBayesianNetwork(edges)
bn.fit(df, estimator=BayesianEstimator, prior_type="BDeu",
       equivalent_sample_size=5,
       state_names={v: [0, 1] for v in df.columns})
assert bn.check_model()
q = VariableElimination(bn)
pz = q.query(["Z"]).values
def p_do(t):
    return sum(q.query(["Y"], evidence={"T": t, "Z": z})
               .values[1] * pz[z] for z in [0, 1])
print(p_do(0), p_do(1), p_do(1) - p_do(0))
```

This adjustment implements $\mathrm{do}(T = t)$ for the sample DAG. Querying evidence $T = t$ alone computes observation.

Sample code: DoWhy effect estimation

```
from dowhy import CausalModel
model = CausalModel(
    data=df, treatment="T", outcome="Y",
    graph="digraph { Z -> T; Z -> Y; T -> Y; }")
estimand = model.identify_effect(
    proceed_when_unidentifiable=False)
print(estimand)
estimate = model.estimate_effect(
    estimand, method_name="backdoor.linear_regression",
    control_value=0, treatment_value=1,
    target_units="ate", confidence_intervals=True)
print(estimate.value)
print(estimate.get_confidence_intervals())
```

DoWhy should identify adjustment for Z. The additive outcome probability makes linear adjustment suitable here.

Sample code: $\mathrm{do}(T = t)$ in DoWhy GCM

```
import networkx as nx
from dowhy import gcm
cat = df.astype(str)  # categorical states "0", "1"
cm = gcm.ProbabilisticCausalModel(nx.DiGraph(edges))
gcm.auto.assign_causal_mechanisms(cm, cat)
gcm.fit(cm, cat)
np.random.seed(42)
risks = []
for t in [0, 1]:
    s = gcm.interventional_samples(
        cm, {"T": lambda x, t=t: str(t)},
        num_samples_to_draw=20000)
    assert (s["T"] == str(t)).all()
    risks.append((s["Y"] == "1").mean())
print(risks, risks[1] - risks[0])
```

The constant function sets T for every sample. Results include mechanism fitting error and Monte Carlo error.

Sample code: refutation checks

```
placebo = model.refute_estimate(
    estimand, estimate,
    method_name="placebo_treatment_refuter",
    placebo_type="permute", num_simulations=100,
    random_seed=42)
subset = model.refute_estimate(
    estimand, estimate,
    method_name="data_subset_refuter",
    subset_fraction=0.8, num_simulations=100,
    random_seed=42)
print(placebo)
print(subset)
```

Expected behavior: placebo effect near 0, subset effect near the original estimate. Report actual outputs and test definitions.

Verification of $\mathrm{do}(T = t)$

Implementation check: compare BN adjustment with the exact values 0.30, 0.60 and $\mathrm{ATE} = 0.30$.
Independent estimate: compare the BN ATE and the DoWhy adjusted ATE on the same dataset and contrast.
Simulation check: use GCM to confirm every intervened T equals t and compare simulated risks with BN adjustment.
Robustness check: run at least two DoWhy refuters and discuss their expected behavior.
Uncertainty: repeat across seeds and report error, spread or confidence intervals. Passing refutation tests cannot prove causality.

UCI tabular data: required source

The main application must use a public, structured tabular dataset from the UCI Machine Learning Repository.

Record the dataset page, identifier or DOI, file version, download date and license.

Document the unit of observation, column meanings, outcome, treatment and measurement times.

Select a question with a defensible intervention and measured potential confounders. UCI membership alone does not justify a causal interpretation.

Keep the synthetic tutoring benchmark as a separate verification experiment. It cannot replace the UCI application.

UCI application example: Student Performance

Dataset: Student Performance, UCI ID 320. Use one subject file and report which one.

Question: Under stated assumptions, how would extra educational support affect the probability of passing?

Treatment: schoolsup (yes/no). Outcome: pass, defined for this project as final grade G3 of at least 10.

Candidate baseline variables: school, age, sex, parental education and previous failures. Justify their timing and role in the DAG.

G1 and G2 are earlier-period grades. Include them only if measured before the chosen intervention.

The dataset does not establish support timing or absence of hidden confounding. Present causal results conditional on these assumptions.

UCI example: reproducible data loading

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
uci = fetch_ucirepo(id=320)
data = pd.concat([uci.data.features,
                  uci.data.targets], axis=1)
print(uci.metadata)
print(uci.variables)
data["support"] = data["schoolsup"].map(
    {"no": 0, "yes": 1})
data["pass"] = (data["G3"] >= 10).astype(int)
# Remove G3 from predictors after creating pass.
# Select pre-intervention covariates using the DAG.
```

Install ucimlrepo first. Check missing values, coding, subject metadata and treatment overlap before modeling.

Baseline and hybrid comparison protocol

Use one basic CBN baseline (B0) and at least two hybrid variants (H1, H2). Keep the population, outcome and intervention contrast fixed.

Use common train/validation/test splits, for example 70%/15%/15%, with at least five paired seeds or splits.

Fit preprocessing and any learned graph on training data only. Use validation data for selection and keep the test set untouched.

Give all variants equivalent measured information. Record any representation or discretization changes as separate components.

Keep the baseline DAG fixed for mechanism or estimator comparisons. For graph variants, rerun identification and report changed assumptions.

Declare tuning budgets, uncertainty methods and hardware. Use group or temporal splits when required.

H1: neural estimation and H2: neural DAG learning

B0: a domain DAG with non-neural CPDs. H1: keep that DAG and replace selected CPDs with neural conditional models.

H2: train neural causal discovery, such as DAG-GNN, on training data with temporal and domain constraints.

For H2, rerun identification and retain the B0 non-neural CPD fitting family and effect-computation procedure, adapting parent or adjustment sets.

Compare B0 vs H1 for neural mechanisms and B0 vs H2 for graph learning. Report identification failures and uncertain edge directions.

A combined H1+H2 model is optional. ChatGPT graph suggestions alone do not satisfy neural causal discovery.

Concrete implementation steps appear on slides 43–47. Required ablations appear on slides 48–51.

Controlled differences between B0, H1 and H2

Component	B0 and H1	H2
DAG	B0: domain DAG. H1: same DAG.	Neural discovery proposes a constrained DAG.
Identification	Same assumptions and adjustment set.	Re-identify under the proposed DAG.
CPD fitting	B0: non-neural. H1: selected neural CPDs.	Same non-neural fitting family as B0.
Comparison	B0 vs H1 isolates neural mechanisms.	B0 vs H2 isolates structure learning.

H2 learns a candidate graph. Observational fit alone does not establish causal directions. Keep the target population and intervention contrast fixed.

Required ablations: one change per variant

Variant	Single change from its parent model	Purpose
B0 / H1 / H2	Primary reference models. Compare H1/B0 and H2/B0.	Mechanism and structure contributions
A1 from H1	Remove dropout or one active neural penalty.	Effect of regularization
A2 from H2	Remove one forbidden-edge constraint. Retrain DAG learning.	Value of domain information
A3 from B0	Replace smoothed CPDs with maximum likelihood.	Sensitivity to sparse counts

A1–A3 are distinct experiments. H1 without its only neural CPD equals B0. Detailed procedures and examples appear on slides 48–51.

Ablation experiment protocol

Define each ablation by its parent, hypothesis and exactly one changed component. A1 pairs with H1, A2 with H2, and A3 with B0.

Use five or more paired runs. Keep eligible rows, split indices, seeds, preprocessing and unchanged settings fixed.

A1 removes one active regularizer. A2 removes one domain constraint while retaining acyclicity. A3 changes CPD smoothing to maximum likelihood.

Keep the same tuning and stopping protocol. Report actual runtime, convergence and any implementation failures.

For A2, rerun identification and flag implausible directions. On UCI data, report effect changes. On synthetic data, report error against known truth.

Show parent and variant results with signed paired changes. Distinguish seed variability from sampling uncertainty. See slides 48–57.

Required result tables and interpretation

Table A: B0, H1 and H2, changed stage, DAG, identification status, adjustment set and estimated effect with uncertainty.

Table B: known-truth benchmark results for B0 and applicable hybrids, absolute causal-effect error, variability and runtime.

Table C: parent model, ablation, single changed component, paired effect change and uncertainty. Add prediction metrics where applicable.

For variants supporting prediction, use a common held-out task. For binary outcomes, report AUROC, macro-F1 and log loss or Brier score. For regression, use MAE and RMSE.

Report at least two DoWhy refutation checks and any estimator compatibility limits. Explain which component helps, fails or changes the assumptions.

Presentation: required scope

Choose a UCI dataset and define the population, variable timing, treatment contrast and causal estimand. See slides 39–42.
Implement B0 with non-neural CPDs, H1 with neural CPDs on the same DAG, and H2 with neural structure learning. See slides 43–47.
Run A1 from H1, A2 from H2 and A3 from B0. Keep one changed component per ablation. See slides 48–51.
Use at least five paired runs, a known-truth benchmark, GCM simulation checks and at least two DoWhy refuters. See slides 52–55.
Present model, benchmark and ablation results with uncertainty, identification status and runtime. See slides 56–57.
Submit reproducible code, presentation and report using the evidence checklist on slides 58–62. The original point totals remain unchanged.

UCI project selection: possible questions

Application	Possible treatment X	Outcome Y	Potential confounders
Education	Study support	Exam success	Prior scores, background
Business	Promotion exposure	Purchase	Prior demand, customer profile
Operations	Preventive maintenance	Failure	Machine age, prior condition
Public policy	Program participation	Employment	Prior income, education

Every application must use a UCI tabular dataset. Select measured variables and justify timing, intervention meaning and adjustment assumptions.

Libraries and their roles

ucimlrepo or an official UCI download: obtain the tabular data and retain metadata.
pgmpy or an equivalent BN library: implement B0 with non-neural CPDs and valid observational and interventional inference.
DoWhy (required): graph-based identification, effect estimation and refutation. Use GCM for intervention simulation checks.
PyTorch or TensorFlow/Keras: implement the selected neural CPDs, estimators or other hybrid modules. Document integration code.
pandas, NumPy and scikit-learn: preprocessing, shared splits and evaluation. Explain any additional discovery or estimation library.
Record tested versions, seeds and hardware. Check that each custom estimator supports the requested validation.

Presentation rubric: foundations (4 points)

Criterion	Evidence for full credit	Points / CLO
Causal representation	Variables, DAG, factorization and assumptions	1 / CLO1, CLO2
Learning and DL roles	Basic CPDs, neural extensions and identification limits	2 / CLO1, CLO2
Inference	Observation, identification and valid intervention queries	1 / CLO1, CLO2

Foundations total: 4 points. Award partial credit for partially correct evidence.

Presentation rubric: application (6 points)

Criterion	Evidence for full credit	Points
Problem and UCI data	Dataset source and defensible causal question	1
Implementation	Runnable B0, H1 and H2 with explicit DL integration	1
Controlled comparison	Same target, paired splits, uncertainty and runtime	1
Causal verification	Known truth and two DoWhy refutation checks	1
Ablation study	Three controlled variants and paired results	1
Conclusions and Q&A	Evidence, identification limits and defense	1

Application: CLO3, CLO4. Presentation total: $4 + 6 = 10$ points.

Presentation: notebook and demonstration

Submit slides and a Colab-compatible notebook with saved outputs and a table of contents.
Include the UCI source, timing, preprocessing, B0 DAG, CPDs, estimand and DoWhy identification output.
Show H1 neural CPDs on the fixed B0 DAG. For H2, show discovery training, domain constraints, proposed DAG and new identification output.
Compare B0 vs H1 and B0 vs H2. Report effects, uncertainty, runtime and three controlled ablations.
Add prediction results where applicable.
Demonstrate a do query, the known-truth benchmark and at least two DoWhy refutation checks.
Explain identification failures.
Include reproducible splits, versions, seeds, citations and an AI-use statement. ChatGPT graph suggestions alone do not satisfy H2.

Report: introduction and problem (3 points)

2.1 Introduction (1 point, CLO1, CLO2): state the UCI question and why a basic CBN is the reference for testing DL extensions.
2.2 Related work (1 point, CLO1, CLO2): review CBNs and DL methods relevant to the selected causal stages.
2.3 Problem statement (1 point, CLO1, CLO2, CLO4): define the population, variables, timing, DAG and estimand.
Example target: P(Y = 1 | do(X = x)) or E[Y | do(X = x1)] minus E[Y | do(X = x0)].
Distinguish causal discovery, formal identification and statistical estimation. State which assumptions each extension requires.

Report: methods (2 points)

2.4 Proposed methods (2 points, CLO3, CLO4).
Specify B0: domain DAG, non-neural CPDs, assumptions, identified formula and intervention computation.
Specify H1: selected neural CPDs on the fixed B0 DAG, neural architecture, loss, regularization and training.
Specify H2: neural discovery method, domain mask, graph threshold, convergence, identification and non-neural CPD fitting.
Define A1 from H1 by removing one regularizer, A2 from H2 by removing one domain constraint, and A3 from B0 using maximum likelihood CPDs.
Give equations, pseudocode, uncertainty methods and handling of failures. See slides 43–55 for implementation requirements.

Report: experimental design (1 point)

2.5 Experimental design (1 point, CLO3, CLO4).
Document UCI selection, measurement timing, treatment overlap and the separate known-truth benchmark.
Compare B0, H1 and H2 on the same population and intervention contrast, using paired splits and declared tuning budgets. Avoid leakage.
Hold the DAG fixed for mechanism comparisons. For graph extensions, report revised assumptions and identification status.
Predefine causal checks, uncertainty, runtime and any applicable prediction metrics. Run three controlled ablations with at least five paired runs.
Specify at least two DoWhy refuters and their expected behavior. Document custom-estimator compatibility and any fallback checks.

Report: results and conclusions (2 points)

2.6 Experimental results (1 point, CLO3, CLO4).
Present B0 versus H1/H2 effects, uncertainty and runtime, plus identification status and three ablations. Add prediction scores where applicable.
On the synthetic benchmark, compare with known truth. On UCI data, report assumption-dependent estimates and robustness checks.
2.7 Conclusions (1 point, CLO4, CLO5).
Use ablations to explain which DL stage contributes, when B0 is sufficient and which assumptions change. Include null or adverse findings.
Discuss hidden confounding, graph uncertainty, overlap, sample size and computational cost. Avoid assuming that a hybrid must outperform B0.

Report: abstract and defense (2 points)

2.8 Abstract (1 point, CLO4, CLO5). State the UCI question, basic CBN baseline, DL extensions, main results, ablations and causal limitations.
2.9 Questions and answers (1 point, CLO1–CLO5). Defend the DAG, identification, DL integration, comparison fairness and refutation interpretation.
Submit a .docx or a PDF compiled from LaTeX, plus source files when requested.
Keep dataset versions, outcome definitions, splits, DAGs and results consistent across the report, notebook and presentation.
The following slide consolidates the report rubric.

Report rubric: 10 points

Section	Points	CLO mapping
2.1 Introduction	1	CLO1, CLO2
2.2 Related work	1	CLO1, CLO2
2.3 Quantitative problem statement	1	CLO1, CLO2, CLO4
2.4 Methods and ablation definitions	2	CLO3, CLO4
2.5 Fair comparison and experiment design	1	CLO3, CLO4
2.6 Prediction, causal and ablation results	1	CLO3, CLO4
2.7 Conclusions	1	CLO4, CLO5
2.8 Abstract	1	CLO4, CLO5
2.9 Questions and answers	1	CLO1–CLO5

Total $= 10$ points. Full credit requires the evidence on slides 30–34. Award partial credit within each section.

Reproducibility and interpretation checklist

Use one documented UCI application and a separate known-truth benchmark. State variable timing and the causal target.
Implement B0, H1 and H2 exactly as defined. Run A1/H1, A2/H2 and A3/B0 comparisons without duplicating the baseline.
Use five or more paired runs, shared eligible information and training-only preprocessing. Preserve an untouched test evaluation.
Verify do queries, run GCM checks and retain two or more supported DoWhy refutation outputs. Document custom-estimator limitations.
Submit per-run results, graphs, CPDs, neural checkpoints, versions, seeds and split indices. See slides 56–59.
Keep causal assumptions separate from predictive scores. Report null findings, identification failures and AI assistance.

References and API documentation

DoWhy documentation: identification, estimation and refutation
https://www.pywhy.org/dowhy/v0.14/
DoWhy: simulating the impact of interventions
https://www.pywhy.org/dowhy/v0.14/user_guide/causal_tasks/what_if/interventions.html
pgmpy: DiscreteBayesianNetwork API
https://pgmpy.org/api/generated/models/pgmpy.models.DiscreteBayesianNetwork.html
Sharma & Kiciman (2020). DoWhy: An End-to-End Library for Causal Inference. arXiv:2011.04216.
The tutoring DAG, CPDs and reference values are a synthetic teaching example.
UCI repository: https://archive.ics.uci.edu/
Student Performance (ID 320): https://archive.ics.uci.edu/dataset/320/student+performance

Implementation guide: required experiment map

B0: domain DAG with non-neural CPDs. H1: the same DAG with selected neural CPDs. H2: a neural-learned DAG with non-neural CPDs.

Primary comparisons: H1 against B0 for mechanism learning, and H2 against B0 for structure learning.

A1: H1 without one regularizer. A2: H2 without one domain constraint. A3: B0 with maximum likelihood CPDs instead of smoothing.

Required evidence: a UCI application, a separate known-truth benchmark, five or more paired runs and two or more DoWhy refutation checks.

Slides 39–47 cover implementation. Slides 48–55 explain ablations and validation. Slides 56–62 specify results and submission evidence.

Causal question: a specification before modeling

Write one sentence naming the population, treatment T, outcome Y and the time of intervention.

Specify control and treatment values, outcome measurement time and the target effect, such as ATE = E[Y under do(T=1)] minus E[Y under do(T=0)].

Create a variable dictionary: meaning, coding, measurement time, proposed causal role and missingness for every selected variable.

List candidate confounders measured before T. Explain the role of any mediator or collider before deciding how to use it.

State assumptions about unmeasured confounding, consistency and overlap. Separate documented facts from assumptions.

Deliverables: the question, dictionary, target formula and an assumptions table shared by the notebook, slides and report.

UCI data: source, timing and eligibility audit

Record the UCI URL, dataset ID or DOI, download date, license, subject file and file checksum. Preserve the original downloaded data.

Report row counts before and after every exclusion. Check duplicates, units, category codes, missing values and class balance.

For Student Performance, define pass from G3, then remove G3 from predictors. Document which subject and sampling unit you use.

Use G1 or G2 only if their measurement precedes the chosen treatment. Explain whether the same student appears more than once.

Inspect treatment counts and outcome counts within important covariate groups. Flag sparse groups and weak overlap.

Deliverables: a source record, eligibility flow, variable-timing table and data-quality summary.

Shared preprocessing and split construction

Create common train, validation and test indices before fitting anything. A 70%/15%/15% split is an example, not a universal requirement.

Use stratification when appropriate. Use group or time splits when observations are dependent or prediction concerns future records.

Fit imputation, category encoding, scaling and discretization on training rows only. Apply the saved transformations elsewhere.

Use the same measured variables and bin definitions for B0, H1 and H2 in the primary comparison. Keep variable-level graph nodes interpretable.

Declare validation budgets, graph thresholds, stopping rules and five or more paired seeds before evaluating the test set.

Save split indices and transformations. Treat unknown categories and missing values using a documented rule.

DAG and identification: required checks

For B0, draw a domain DAG and justify each edge direction. Verify that the graph is acyclic and that temporal order is defensible.

In DoWhy, supply the graph, treatment and outcome. Save the identification output and the chosen estimand or adjustment set.

Use the identification criterion appropriate to the graph. For a simple observed-confounder design, justify a valid backdoor set.

Check that the implementation follows the identified formula. Conditioning on T alone generally differs from intervention.

For H2 and A2, repeat identification after graph learning. Record changed assumptions, adjustment sets and any failure.

If identification fails, report the failure and missing assumptions. Software success under a supplied graph does not verify that graph.

B0: basic CBN implementation workflow

1. Fix a justified DAG and variable states. Start with a small graph whose conditional tables have adequate support.

2. Fit non-neural CPDs on training data. For discrete variables, use Bayesian smoothing and declare its prior and strength.

3. Check every CPD: expected shape, finite entries, probabilities summing to one and coverage of declared states.

4. Compute observational predictions and intervention estimates using valid adjustment or truncated factorization.

5. On the tutoring benchmark, verify the two intervention risks and ATE against the population reference values.

6. Save the DAG, fitted CPDs, estimator settings, identification output and observed versus interventional results.

H1: neural CPDs on the fixed B0 graph

Start with one replaced node, preferably Y. Keep the B0 DAG, state definitions, data splits and all other CPDs fixed.

Give the neural model only the selected node’s parents as inputs. For binary Y, use a probability output and binary cross-entropy.

Document hidden layers, widths, activations, dropout, optimizer, learning rate, batch size and parameter count.

Train on training rows. Use validation loss for early stopping and save the selected checkpoint. Keep the test set untouched.

For a discrete parent space, evaluate the network at every parent configuration to form a normalized neural CPD table.

Insert that CPD into a copy of B0. Run the same do-query procedure and compare H1 against B0.

H1: a concrete intervention calculation

For the tutoring DAG, train p_theta(Y=1 | T,Z). Keep the B0 marginal P(Z) and all other CPDs fixed.

Set T=0 and evaluate the network for Z=0 and Z=1. Weight these probabilities by the same B0 P(Z).

Repeat with T=1. Subtract the two intervention risks to obtain the H1 ATE.

Formula: risk_H1(t) = sum over z of p_theta(Y=1 | t,z) times P_B0(z).

B0 uses its non-neural outcome CPD in the same calculation. This comparison changes only the outcome mechanism.

For a different DAG, derive its valid formula first. If mediators exist, simulate or marginalize them under intervention.

H2: neural causal discovery workflow

Choose a neural structure-learning implementation, such as DAG-GNN, and cite the method, code version and modifications.

Use the same measured variables as B0. Train only on training data with a loss appropriate to their variable types.

Apply temporal and domain constraints during graph learning. Declare edge thresholds, sparsity settings and convergence checks.

Save the learned adjacency weights and final DAG. Document thresholding, any cycle repair and uncertain edge directions.

Run DoWhy identification on the proposed DAG. Fit non-neural CPDs using the same estimator family and prior strength as B0.

Compute the same target effect and compare H2 against B0. ChatGPT graph suggestions alone do not satisfy H2.

H2: domain constraints and adjacency masks

List forbidden edges with reasons. Example: a final exam result Y cannot cause prior preparation Z.

Define a fixed convention, such as W[i,j] meaning edge i to j. Build an allowed-edge mask M with zeros for forbidden edges.

Use effective adjacency W_eff = M multiplied elementwise by W in the model and its graph constraints. Keep diagonal entries zero.

If the library offers a forbidden-edge interface, use it. Otherwise document and verify the masking changes to the implementation.

Keep the acyclicity constraint active. Verify that forbidden weights remain zero and the final graph is acyclic.

Save the constraint list, mask, node order, learned weights and a constraint-compliance check.

A1: regularization ablation from H1

Parent: the full H1 model. First choose an H1 regularizer that is active, such as dropout with a declared nonzero rate.

Single change: set that dropout rate to zero. If H1 uses another regularizer, remove exactly that one penalty instead.

Keep the DAG, replaced node, layer widths, activation functions, inputs, data splits and other training settings fixed.

Keep the same validation stopping rule and maximum budget. Do not retune other hyperparameters for the primary ablation.

Across paired runs, compare training and validation loss, held-out prediction and the estimated causal effect.

H1 versus B0 already tests neural removal when H1 has one neural CPD. A duplicate B0 run is not a new ablation.

A2: one removed domain constraint from H2

Parent: full H2, which forbids Y to Z because Y occurs later than Z. Select this one prohibition before inspecting results.

Single change: allow that candidate edge by changing its mask entry from zero to one. Keep all other constraints, including acyclicity.

Retrain graph learning with the same split, seed, threshold and tuning settings. Removing a ban does not force the edge to appear.

Compare adjacency weights, learned edges and direction stability. Then rerun identification and fit the same non-neural CPD family.

Report effect changes only when interpretable. Label a graph contradicting time order as a sensitivity result.

If the DAG stays unchanged, report that null finding. If no removable constraint exists, revise the H2 design before running A2.

A3: smoothing versus maximum likelihood

Parent: discrete B0 with smoothed CPDs. Single change: estimate CPDs by maximum likelihood from the same training rows.

Keep the DAG, node states, bins, missing-data treatment, queries and inference procedure fixed. Refit parameters only.

Example group T=1, Z=0: five students, five passes and zero failures. The maximum likelihood pass probability is 5/5 = 1.

With an illustrative add-one prior for each outcome, the smoothed probability is (5+1)/(5+2) = 6/7, about 0.857.

This add-one example explains smoothing. BDeu distributes its declared prior strength across states and parent configurations.

Compare sparse-cell probabilities, predictive calibration, effect estimates and variability. Improvement in either direction is possible.

A3: pgmpy fitting pattern and checks

Create two fresh DiscreteBayesianNetwork objects with identical edges and nodes. Use one shared state_names mapping.

B0 fit: estimator=BayesianEstimator, prior_type="BDeu", equivalent_sample_size=5 as an example declared setting.

A3 fit: estimator=MaximumLikelihoodEstimator. Use the same training data and state_names. Omit the prior arguments.

Run check_model() for both. Compare CPD dimensions, state order, normalization, zero entries and unsupported parent configurations.

Recompute the same prediction and do queries for both fitted models. Count any infinite log loss from zero-probability predictions.

Report numerical stabilization separately from probability smoothing. Preserve the raw CPDs and disclose scoring conventions.

Paired runs, tuning and uncertainty

Use at least five paired seeds or splits. For each run, all models and ablations share eligible rows, outcomes and split indices.

Save the full run configuration. Compare A1 to H1, A2 to H2 and A3 to B0 within the same run.

Report raw run values and signed paired differences. Summarize with mean and standard deviation or an explicitly justified interval.

Variation across training seeds measures algorithmic stability. It is not automatically a confidence interval for the population ATE.

For sampling uncertainty, choose and document a valid resampling or inference method. Refit learned components when needed.

Separate intervals conditional on one fixed DAG from uncertainty that includes graph learning. Report timing and hardware consistently.

Known-truth benchmark: expected outputs

Use the tutoring model on slides 3–14 separately from the UCI application. Generate fresh data with recorded seeds and sample sizes.

Population targets: observational risks 0.18 and 0.72, intervention risks 0.30 and 0.60, and causal ATE 0.30.

Run B0 and H1 on the known DAG. Run H2 and A2 on observed samples while keeping the true DAG only for evaluation.

For each applicable variant, report estimated risks, ATE, absolute error against 0.30 and variability across paired runs.

For graph learning, also report missing, extra or reversed edges using a clearly defined graph metric.

Small samples, failed graph recovery or worse neural results are valid findings. Report them and explain plausible causes.

DoWhy refutation: execution and interpretation

Run at least two checks, including a permuted-treatment placebo and a data-subset refuter. Record simulation counts and seeds.

For the placebo, state that permuting treatment should remove the original treatment association. Inspect the returned effect and test definition.

For the subset check, compare estimates after repeated subsampling. Explain the subset fraction and expected variability.

Run the checks on the supported DoWhy estimate. State which estimator and graph each result actually validates.

For a custom neural estimate, verify refitter compatibility. Otherwise implement and label an analogous retraining check separately.

Retain the required DoWhy outputs for a supported reference estimate. Passing checks does not prove the DAG or absence of confounding.

Intervention simulation and inference checks

Use DoWhy GCM for a separate intervention simulation check. Record the assigned mechanisms and compare them with the CBN mechanisms.

For do(T=t), generate samples with T fixed to t for every row. Confirm that the intervention removes the original T mechanism.

Estimate the outcome risk or mean from the simulated sample. Increase simulation size to inspect Monte Carlo variability.

When comparing exact CBN queries with GCM, match mechanisms if feasible. Otherwise explain that model fitting can also change the result.

For neural CPDs, use categorical sampling from valid neural probabilities or integrate an explicitly tested custom mechanism.

Individual counterfactual claims require an appropriate structural model and assumptions about noise. Treat them as optional extensions.

Result table A: baseline and hybrid comparisons

One row per model and run: B0, H1 and H2. Include sample counts and the shared treatment contrast.

Columns: graph identifier, changed component, identification status, adjustment set and plausibility flags.

Causal columns: risk or mean under each intervention, ATE, interval method and whether the interval conditions on a fixed DAG.

Prediction columns, where supported: AUROC, macro-F1 and Brier score or log loss for binary Y. Use MAE and RMSE for regression.

Report training time, effect-query time, hardware and graph-learning time separately where material.

Provide individual run values plus a summary. Use N/A with a reason for unavailable outputs or identification failures.

Result tables B and C: truth and ablations

Table B, synthetic truth: model, sample size, run, estimated ATE, true ATE=0.30, absolute error and graph errors where applicable.

Table C, ablations: parent, variant, one changed setting, parent result, variant result and their paired difference.

Required pairings: H1 with A1, H2 with A2, and B0 with A3. Show B0 with H1 and B0 with H2 separately.

Define the sign convention, such as delta = variant minus parent. Explain whether lower or higher is favorable for each metric.

On UCI data, report effect change without labeling it error. Add identification and plausibility status for graph variants.

Report all completed runs, failures, exclusions and runtime. Use ablations to support claims about the changed component only.

Notebook structure and reproducible artifacts

Sections: setup, data audit, split creation, DAG and identify, B0, H1, H2, A1–A3, synthetic verification, refutation and result tables.

Use one configuration object for seeds, data paths, node order, state names, priors, neural settings, masks and graph thresholds.

Save split indices, transformations, CPDs, neural checkpoints, adjacency weights, final graphs and identification text.

Save raw per-run results and tables in machine-readable files. Record exact package versions and the repository revision.

Restart a fresh Colab runtime and execute all cells in order. Include commands for setup and a declared expected runtime.

Retain saved outputs in the submitted notebook. Document downloads, external code and any manual steps.

Presentation demonstration: concrete sequence

1. State the UCI question, variable timing, B0 DAG and the assumptions needed to interpret the effect.
2. Show the identified formula, B0 CPDs and one observational probability alongside its do-query counterpart.
3. Show exactly which CPD H1 replaces. For H2, show the constraints, learned graph and identification output.
4. Change the intervention value and recompute risks. Explain why the operation follows the causal model.
5. Present paired B0/H1/H2 comparisons, A1–A3 results, synthetic truth checks and DoWhy refutation outputs.
6. Explain one failure, null result or limitation. Each student must be able to trace a displayed result to the code.

Report sections: evidence for each requirement

2.1–2.3: motivate the UCI question, review CBN and selected neural methods, then define variables, timing, DAG and estimand.

2.4: specify B0/H1/H2 mathematically, give neural and graph-learning settings, and define A1–A3 with their parent models.

2.5: document data eligibility, preprocessing, paired splits, tuning, uncertainty, benchmark design and refutation procedures.

2.6: include model, synthetic-truth and ablation tables, raw-run variability, identification failures and runtime.

2.7–2.8: conclude from the measured results and summarize the main finding with causal limitations. Include null findings.

2.9: defend modeling and evaluation choices. Keep citations, AI disclosure and all definitions consistent across submissions.

Common failures and required responses

No overlap or unsupported cells: report affected groups, quantify the problem and narrow the causal target only with justification.

Unidentified or temporally implausible H2 graph: report it explicitly. Avoid presenting a numerical result as a validated causal effect.

Cycles after thresholding: apply a declared repair rule consistently or report graph-learning failure. Save both raw and final graphs.

Zero ML probabilities or absent parent states: document the estimator convention. Keep any metric clipping separate from CPD smoothing.

Neural overfitting, unstable effects or poor benchmark recovery: show losses and paired runs. Do not suppress unfavorable results.

A failed package integration or refuter: provide the actual limitation, a supported reference check and a clearly labeled alternative.

Optional extensions and implementation references

Optional H1+H2: use neural CPDs on the learned H2 graph. Compare with both parents to separate graph and mechanism contributions.

Other optional DL roles include conditional density models, learned representations and approximate inference. State additional assumptions.

DAG-GNN paper and code: arxiv.org/abs/1904.10098 and github.com/fishmoon1234/DAG-GNN.

Neural NOTEARS alternative: github.com/xunzheng/notears. Use its nonlinear neural implementation if selecting this option.

CPD fitting and smoothing: pgmpy.org/guides/parameter_estimation.html. DoWhy documentation covers identify, refute and GCM.

Required core remains B0, H1, H2, A1–A3, paired runs, the UCI study, synthetic verification and documented refutation checks.